

4회차 과제

17기 김지우

01 **조사과제**

02 **실습과제**

03 **마무리**

포인터 메모리상의 주소값

```
#include <stdio.h>

int main()
{
    int num = 10;
    int *ptr = &num;

    printf("1. 변수 num의 값: %d\n", num);
    printf("2. 변수 num의 주소 (&num): %p\n", &num);
    printf("3. 포인터 ptr이 가진 값 (주소): %p\n", ptr);
    printf("4. 포인터 ptr이 가리키는 실제 값 (*ptr): %d\n", *ptr);

    *ptr = 20;
    num = 50;
    printf("\n--- 포인터로 값을 변경한 후 ---\n");
    printf("5. 변경된 ptr의 값: %d\n", *ptr);

    return 0;
}
```

& = 변수가 할당된 메모리의 시작 주소

* = 포인터가 가리키는 주소값

*ptr은 num과 같다.

쉽게 말해서 *ptr은 ptr이 가리키는 주소
num은 실질적인 값.

```
1. 변수 num의 값 : 10
2. 변수 num의 주소 (&num): 0x16d386f78
3. 포인터 ptr이 가진 값 (주소): 0x16d386f78
4. 포인터 ptr이 가리키는 실제 값 (*ptr): 10

--- 포인터로 값을 변경한 후 ---
5. 변경된 ptr의 값 : 50
```

Call by Value(값에 의한 호출)

```
1  #include <stdio.h>
2
3  void change(int n)
4  {
5      n = 100;
6  }
7
8  int main()
9  {
10     int a = 10;
11     change(a);
12     printf("%d\n", a);
13 }
14
```

- 변수의 '값'만 복사해서 전달
- 인자를 주소말고 값만 전달함.
- change 함수에서 아무리 값을 바꿔도 원본에는 영향을 안 줌.

PROBLEMS OUTPUT DEBUG CONSOLE

```
● jiwoo@gimjiuui-MacBookAir knock % g
● jiwoo@gimjiuui-MacBookAir knock % .
10
○ jiwoo@gimjiuui-MacBookAir knock %
```

Call by Reference (참조에 의한 호출)

```
3 void change(int *n)
4 {
5     *n = 100;
6 }
7
8 int main()
9 {
10     int a = 10;
11     change(&a);
12     printf("%d\n", a);
13 }
```

PROBLEMS OUTPUT DEBUG CONSOLE

● jiwoo@gimjiuui-MacBookAir knock
100

- 변수의 주소를 전달
- 인자로 주소를 전달함.
- 포인터로 매개변수를 받아서 n 이 가르키는 주소의 값을 바꿈.
- 원본 값도 바뀜

문자열 (null 문자로 끝내기)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str1[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
6      char str2[5] = {'H', 'e', 'l', 'l', 'o'};
7      char str3[10] = {'H', 'e', 'l', 'l', 'o'};
8
9      printf("%s\n", str1);
10     printf("%s\n", str2);
11     printf("%s\n", str3);
12
13     return (0);
14 }
```

- ‘\0’ 은 문자열의 끝을 알려줌.
- 선언한 배열보다 작은 개수의 값을 넣으면
저절로 남은 자리는 모두 널 문자로 채워줌

```
Hello
Hello@J?Hello
Hello
```

문자열 (선언 방식(포인터 or 배열로))

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str[] = "Apple";
6      char *ptr = "Banana";
7
8      str[0] = '0';
9      printf("1. 배열 수정 결과: %s\n", str);
10     ptr = "Cherry";
11     printf("2. 포인터 교체 결과: %s\n", ptr);
12     return (0);
13 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
● jiwoo@gimjiuui-MacBookAir knock % ./a.out
1. 배열 수정 결과: 0pple
2. 포인터 교체 결과: Cherry
○ jiwoo@gimjiuui-MacBookAir knock %
```

- 배열은 내용을 쉽게 바꿀 수 있다.
- 포인터는 내용을 바꿀 순 없지만, 다른 곳을 쉽게 가르킬 수 있다.
- 기존에 가르키던 포인터 주소는 미아가 됨.

배열, 문자열, 포인터 차이

- 배열은 데이터 수정이 가능
- 문자열은 마지막에 null 문자가 있고, 수정이 불가능.
- 포인터는 주소값을 쉽게 변경할 수 있음.
- 배열은 스택에 저장 (알맹이를 수정 가능)
- 문자열은 데이터에 저장 (읽기 전용)
- 포인터는 스택 (알맹이 자체를 변경 가능)

문자열 표준 함수(strlen)

```
1  #include <stdio.h>
2
3  int ft_strlen(char *s)
4  {
5      int len;
6
7      len = 0;
8      if (!s)
9          return (0);
10     while (s[len] != 0)
11         len++;
12     return (len);
13 }
14
15 int main()
16 {
17     char *c = "hello\n";
18     int i = ft_strlen(c);
19     printf("%d\n", i);
20 }
21
```

- strlen은 길이를 반환하는 함수
- hello\n 총 6개
- '\0' 만나기 전까지 숫자를 셈

PROBLEMS OUTPUT DEBUG CONSOL

● jiwoo@gimjiuui-MacBookAir knock % .
6

문자열 표준 함수(strcmp)

```
1  #include <stdio.h>
2
3  int ft_strcmp(char *s1, char *s2)
4  {
5      while (*s1 && (*s1 == *s2))
6      {
7          s1++;
8          s2++;
9      }
10     return ((unsigned char)*s1 - (unsigned char)*s2);
11 }
12
13 int main()
14 {
15     char *c = "hello\n";
16     char *s = "hello0\n";
17     int i = ft_strcmp(c, s);
18     printf("%d\n", i);
19 }
```

- 두 문자열이 같은지 비교
- 0 : 두 문자열이 완전히 일치함.
- 음수: 첫 번째로 다른 문자의 아스키 값이 $s1 < s2$ 인 경우.
- 양수: 첫 번째로 다른 문자의 아스키 값이 $s1 > s2$ 인 경우.

문자열 표준 함수(strcpy)

```
3 char *ft_strcpy(char *dest, char *src)
4 {
5     int i = 0;
6
7     while (src[i] != '\0')
8     {
9         dest[i] = src[i];
10        i++;
11    }
12    dest[i] = '\0';
13    return (dest);
14 }
15
16 int main()
17 {
18     char c[] = "hello\n";
19     char a[100];
20     ft_strcpy(a, c);
21     printf("%s\n", a);
22     return 0;
23 }
24
```

- 문자열을 대상 메모리로 복사합니다.
- 복사가 완료된 목적지 주소(dest)를 반환합니다.
-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
jiwoo@gimjiuui-MacBookAir knock % gcc main.c
jiwoo@gimjiuui-MacBookAir knock % ./a.out
hello
jiwoo@gimjiuui-MacBookAir knock %
```

```

3  int main()
4  {
5      int arr[] = { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 };
6      int size = sizeof(arr);
7      int cnt = sizeof(arr) / sizeof(int);
8
9      int sum = 0;
10     int max = arr[0];
11     int min = arr[0];
12
13     printf("배열의 크기 : %d\n", size);
14     printf("배열의 원소 개수 : %d\n", cnt);
15     printf("배열의 원소 : ");
16
17     int i = 0;
18     while (i < cnt)
19     {
20         printf("%d ", arr[i]);
21         sum += arr[i];
22
23         if (arr[i] > max)
24             max = arr[i];
25
26         if (arr[i] < min)
27             min = arr[i];
28
29         i++;
30     }
31     printf("\n");
32     printf("배열의 전체 합 : %d\n", sum);
33     printf("최댓값 : %d\n", max);
34     printf("최솟값 : %d\n", min);
35
36     return (0);
37 }
38

```

- 배열의 크기는 40, int 는 4바이트 이니까, 크기 / 바이트를 하면 원소의 개수를 알 수 있음.
- while 문 돌면서 원소를 출력하고, 최대,최솟값을 저장

```

배열의 크기 : 40
배열의 원소 개수 : 10
배열의 원소 : 2 4 6 8 10 12 14 16 18 20
배열의 전체 합 : 110
최댓값 : 20
최솟값 : 2

```

```

2
3  int fibonacci(int n)
4  {
5      if (n <= 0)
6          return 0;
7      else if (n == 1)
8          return 1;
9      else
10         return fibonacci(n - 1) + fibonacci(n - 2);
11 }
12
13 int main()
14 {
15     int num;
16     int i = 0;
17
18     printf("정수를 입력하세요 : ");
19     scanf("%d", &num);
20
21     while (i < num)
22     {
23         printf("%d ", fibonacci(i));
24         i++;
25     }
26     printf("\n");
27     return (0);
28 }

```

- 함수에 i 만큼 매개변수가 들어오는데 -1, -2 씩 재귀함수를 돌면서 값을 반환해준다.
- 예를 들어 n 이 3 이라면 왼쪽 재귀는 2, 오른쪽 재귀는 1 재귀를 돌면 왼쪽 재귀는 다시 재귀를 돌면서, 1,0 그래서 반환값은 1, 오른쪽 재귀는 리턴 1 이니까 두개 합해서 2 가 나온다.


```

int main()
{
    int row, col;
    int matrix1[10][10], matrix2[10][10], result[10][10];

    printf("행렬의 행과 열 크기를 입력하세요 (최대 10): ");
    scanf("%d %d", &row, &col);

    printf("첫번째 행렬 입력\n");
    printf("행렬의 요소를 입력하세요 :\n");
    put_matrix(matrix1, row, col);

    printf("\n두번째 행렬 입력\n");
    printf("행렬의 요소를 입력하세요 :\n");
    put_matrix(matrix2, row, col);

    printf("\n첫 번째 행렬 :\n");
    print_matrix(matrix1, row, col);

    printf("\n두 번째 행렬 :\n");
    print_matrix(matrix2, row, col);

    printf("\n행렬 덧셈 결과 :\n");
    matrices_plus(matrix1, matrix2, result, row, col);
    print_matrix(result, row, col);

    printf("\n행렬 뺄셈 결과 :\n");
    matrices_minus(matrix1, matrix2, result, row, col);
    print_matrix(result, row, col);

    return (0);
}

```

```

void put_matrix(int matrix[10][10], int row, int col)
{
    int i = 0;
    while (i < row)
    {
        int j = 0;
        while (j < col)
        {
            printf("matrix[%d][%d]: ", i, j);
            scanf("%d", &matrix[i][j]);
            j++;
        }
        i++;
    }
}

void print_matrix(int matrix[10][10], int row, int col)
{
    int i = 0;
    while (i < row)
    {
        int j = 0;
        while (j < col)
        {
            printf("%d ", matrix[i][j]);
            j++;
        }
        printf("\n");
        i++;
    }
}

```

```

int main()
{
    int row, col;
    int matrix1[10][10], matrix2[10][10], result[10][10];

    printf("행렬의 행과 열 크기를 입력하세요 (최대 10): ");
    scanf("%d %d", &row, &col);

    printf("첫번째 행렬 입력\n");
    printf("행렬의 요소를 입력하세요 :\n");
    put_matrix(matrix1, row, col);

    printf("\n두번째 행렬 입력\n");
    printf("행렬의 요소를 입력하세요 :\n");
    put_matrix(matrix2, row, col);

    printf("\n첫 번째 행렬 :\n");
    print_matrix(matrix1, row, col);

    printf("\n두 번째 행렬 :\n");
    print_matrix(matrix2, row, col);

    printf("\n행렬 덧셈 결과 :\n");
    matrices_plus(matrix1, matrix2, result, row, col);
    print_matrix(result, row, col);

    printf("\n행렬 뺄셈 결과 :\n");
    matrices_minus(matrix1, matrix2, result, row, col);
    print_matrix(result, row, col);

    return (0);
}

```

```

void matrices_plus(int a[10][10], int b[10][10], int result[10][10], int row, int col)
{
    int i = 0;
    while (i < row)
    {
        int j = 0;
        while (j < col)
        {
            result[i][j] = a[i][j] + b[i][j];
            j++;
        }
        i++;
    }
}

void matrices_minus(int a[10][10], int b[10][10], int result[10][10], int row, int col)
{
    int i = 0;
    while (i < row)
    {
        int j = 0;
        while (j < col)
        {
            result[i][j] = a[i][j] - b[i][j];
            j++;
        }
        i++;
    }
}

```

- 배열 그 자체를 매개변수로 넘길 때, 주소값을 넘기기 때문에 원본 값이 바뀜

행렬의 행과 열 크기를 입력하세요 (최대 10): 2 2

첫 번째 행렬 입력

행렬의 요소를 입력하세요 :

matrix[0][0]: 1

matrix[0][1]: 2

matrix[1][0]: 3

matrix[1][1]: 4

두 번째 행렬 입력

행렬의 요소를 입력하세요 :

matrix[0][0]: 5

matrix[0][1]: 6

matrix[1][0]: 7

matrix[1][1]: 8

첫 번째 행렬 :

1 2

3 4

두 번째 행렬 :

5 6

7 8

행렬 덧셈 결과 :

6 8

10 12

행렬 뺄셈 결과 :

-4 -4

-4 -4

jiwoo@gimiiuui-MacBookAir knock %

THANK YOU